

- WHU OS Lab 5 实验报告：用户虚拟内存管理与系统调用实现

- 一、实验目的

- 二、实验原理

- 2.1 用户地址空间布局

- 2.2 mmap 区域管理

- 2.2.1 mmap_region 结构

- 2.2.2 mmap 仓库机制

- 2.3 虚拟内存操作

- 2.3.1 页表销毁

- 2.3.2 页表拷贝

- 2.3.3 堆空间管理

- 2.4 内存映射系统调用

- 2.4.1 mmap 系统调用

- 2.4.2 munmap 系统调用

- 2.5 用户态与内核态数据拷贝

- 2.5.1 copyin (用户态 → 内核态)

- 2.5.2 copyout (内核态 → 用户态)

- 2.5.3 copyin_str (拷贝字符串)

- 三、实验内容

- 3.1 实现 mmap 区域管理 (mmap.c)

- 3.1.1 mmap_init()

- 3.1.2 mmap_region_alloc()

- 3.1.3 mmap_region_free()

- 3.2 实现用户虚拟内存管理 (uvm.c)

- 3.2.1 destroy_pgtbl()

- 3.2.2 uvm_destroy_pgtbl()

- 3.2.3 uvm_copy_pgtbl()

- 3.2.4 uvm_mmap()

- 3.2.5 uvm_munmap()

- 3.2.6 堆管理函数

- 3.2.7 数据拷贝函数

- 3.3 实现系统调用 (syscall.c 和 sysfunc.c)

- 3.3.1 syscall() - 系统调用分发器

- 3.3.2 sys_brk() - 堆管理

- 3.3.3 sys_mmap() - 内存映射

- 3.3.4 sys_munmap() - 取消映射

- 3.4 修改进程结构

- 3.5 头文件修改
- 四、实验结果与测试
 - 4.1 编译结果
 - 4.2 功能测试
 - 4.2.1 mmap 仓库测试
 - 4.2.2 堆管理测试
 - 4.2.3 mmap/munmap 测试
 - 4.2.4 数据拷贝测试
- 五、实验心得
 - 5.1 技术收获
 - 5.2 遇到的问题及解决
 - 5.3 改进方向
- 六、实验总结

WHU OS Lab 5 实验报告：用户虚拟内存管理与系统调用实现

一、实验目的

本实验旨在实现完整的用户虚拟内存管理系统和系统调用机制，主要包括：

1. 实现内存映射区域（mmap region）的管理机制
2. 实现用户虚拟内存（UVM）管理功能
3. 实现堆空间的动态分配和回收
4. 实现内存映射（mmap/munmap）系统调用
5. 实现用户态与内核态之间的数据拷贝
6. 完善页表管理和销毁机制
7. 理解现代操作系统的内存管理机制

二、实验原理

2.1 用户地址空间布局

在 RISC-V SV39 虚拟内存系统中，用户进程的地址空间布局如下：

```

MAXVA (256GB)
|
+-- TRAMPOLINE      (最高地址 - PGSIZE)
|   [跳板页，用于特权级切换]
|
+-- TRAPFRAME       (TRAMPOLINE - PGSIZE)
|   [陷阱帧，保存用户态寄存器]
|
+-- KSTACK(0)       (TRAPFRAME - 2*PGSIZE)
|   [内核栈]
|
~~ 可用于 mmap 的空间 ~~
|
+-- User Stack      (heap_top + ustack_pages * PGSIZE)
|   [用户栈，向下增长]
|
+-- Heap            (heap_top)
|   [堆空间，向上增长]
|
+-- Code & Data     (从 PGSIZE 开始)
|   [代码段和数据段]
|
+-- NULL Page       (0 ~ PGSIZE)
|   [保留页，不映射]

```

2.2 mmap 区域管理

2.2.1 mmap_region 结构

```

typedef struct mmap_region {
    uint64 begin;           // 起始地址
    uint32 npages;          // 管理的页面数量
    struct mmap_region* next; // 链表指针
} mmap_region_t;

```

每个进程维护一个 mmap 区域链表，用于跟踪所有通过 **mmap** 系统调用分配的内存区域。

2.2.2 mmap 仓库机制

为了高效管理 mmap_region 结构，实现了一个预分配的仓库 (pool)：

- 预分配 256 个 **mmap_region_node_t** 结构
- 使用单向链表组织空闲节点
- 使用自旋锁保护并发访问

- `list_head` 节点保留，不会被分配出去

2.3 虚拟内存操作

2.3.1 页表销毁

页表销毁需要递归释放三级页表结构：

1. Level 2-1: 中间级页表

- 检查 PTE 是否有效
- 如果是页表指针 (`PTE_CHECK` 为真)，递归释放子页表
- 释放页表本身占用的物理页（内核页）

2. Level 0: 叶子级页表

- 释放数据页（用户页）

3. 特殊处理: trapframe 和 trampoline

- 先清除这两个特殊页的映射
- 避免被错误释放

2.3.2 页表拷贝

进程 fork 时需要拷贝整个地址空间：

1. 代码段和数据段 (`PGSIZE ~ heap_top`)
2. 用户栈 (`heap_top ~ heap_top + ustack_pages * PGSIZE`)
3. mmap 区域: 遍历 mmap 链表，逐个拷贝

对每个页面：

- 分配新的物理页
- 拷贝数据 (`memmove`)
- 在新页表中建立映射

2.3.3 堆空间管理

堆扩展 (`uvm_heap_grow`):

```
旧堆顶对齐 = PG_ROUND_UP(heap_top)
新堆顶对齐 = PG_ROUND_UP(new_heap_top)
```

```
if (新堆顶对齐 > 旧堆顶对齐):
    分配并映射新增的页面
```

堆收缩 (`uvm_heap_ungrow`):

```
新堆顶对齐 = PG_ROUND_UP(new_heap_top)
旧堆顶对齐 = PG_ROUND_UP(heap_top)
```

```
if (新堆顶对齐 < 旧堆顶对齐):
    解除并释放多余的页面
```

2.4 内存映射系统调用

2.4.1 mmap 系统调用

功能：在进程地址空间中创建新的内存映射

参数：

- `start`: 起始地址（0 表示自动分配）
- `len`: 长度（必须页对齐）

实现步骤：

1. 参数验证（页对齐检查）
2. 地址分配：
 - 如果 `start = 0`，从用户栈之上寻找空闲区域
 - 如果指定地址，检查是否与现有区域冲突
3. 创建 `mmap_region` 节点并插入链表
4. 分配物理页并建立页表映射

2.4.2 munmap 系统调用

功能：取消内存映射

参数：

- **start**: 起始地址
- **len**: 长度

实现步骤:

1. 参数验证
2. 在 mmap 链表中查找重叠区域
3. 处理区域分裂:
 - 完全包含: 删除整个区域
 - 部分重叠: 调整区域大小
 - 中间释放: 分裂成两个区域
4. 尝试合并相邻区域
5. 解除页表映射并释放物理页

2.5 用户态与内核态数据拷贝

2.5.1 copyin (用户态 → 内核态)

```
while (len > 0):  
    1. 获取虚拟地址对应的 PTE  
    2. 提取物理地址  
    3. 计算当前页内可拷贝的字节数  
    4. 执行 memmove  
    5. 移动到下一页
```

2.5.2 copyout (内核态 → 用户态)

与 copyin 类似，但方向相反。

2.5.3 copyin_str (拷贝字符串)

特点:

- 遇到 **\0** 终止符时停止
- 最多拷贝 **maxlen** 字节
- 需要逐字节检查

三、实验内容

3.1 实现 mmap 区域管理 (mmap.c)

3.1.1 mmap_init()

初始化 mmap 区域仓库：

```
void mmap_init()
{
    // 初始化自旋锁
    spinlock_init(&list_lk, "mmap_list");

    // 初始化链表：将所有 mmap_region_node 连成单向链表
    list_head = &list_mmap_region_node[0];

    for (int i = 0; i < N_MMAP - 1; i++) {
        list_mmap_region_node[i].next = &list_mmap_region_node[i + 1];
    }
    list_mmap_region_node[N_MMAP - 1].next = NULL;
}
```

设计要点：

- 使用数组预分配所有节点，避免动态内存分配
- list_head 保留不分配，简化链表操作
- 自旋锁保证多核环境下的线程安全

3.1.2 mmap_region_alloc()

从仓库分配一个 mmap_region：

```
mmap_region_t* mmap_region_alloc()
{
    spinlock_acquire(&list_lk);

    // list_head 保留，从第二个节点开始分配
    if (list_head->next == NULL) {
        spinlock_release(&list_lk);
        panic("mmap_region_alloc: out of mmap regions");
    }

    // 取出 list_head 的下一个节点
    mmap_region_node_t* node = list_head->next;
    list_head->next = node->next;

    spinlock_release(&list_lk);
}
```

```
return &(node->mmap);
}
```

设计要点:

- 使用头插法, $O(1)$ 时间复杂度
- 分配失败时 **panic**, 确保内存耗尽时能及时发现

3.1.3 mmap_region_free()

归还 mmap_region 到仓库:

```
void mmap_region_free(mmap_region_t* mmap)
{
    // 通过结构体成员偏移计算外层结构地址
    mmap_region_node_t* node = (mmap_region_node_t*)mmap;

    spinlock_acquire(&list_lk);

    // 将节点插入链表头部
    node->next = list_head->next;
    list_head->next = node;

    spinlock_release(&list_lk);
}
```

设计要点:

- 利用 `mmap_region_t` 是第一个成员的特性，直接类型转换
- 头插法回收，保持简单高效

3.2 实现用户虚拟内存管理 (uvm.c)

3.2.1 destroy_pgtbl()

递归销毁页表:

```
static void destroy_pgtbl(pgtbl_t pgtbl, uint32 level)
{
    if (level > 0) {
        // 中间级页表
        for (int i = 0; i < 512; i++) {
            pte_t pte = pgtbl[i];
```

```

        if (pte & PTE_V) {
            if (PTE_CHECK(pte)) {
                // 这是页表指针，递归释放
                uint64 child = PTE_TO_PA(pte);
                destroy_pgtbl((pgtbl_t)child, level - 1);
                pmem_free(child, true); // 释放页表页（内核页）
            }
        }
    }
} else {
    // 叶子级：释放数据页
    for (int i = 0; i < 512; i++) {
        pte_t pte = pgtbl[i];
        if ((pte & PTE_V) && !PTE_CHECK(pte)) {
            uint64 pa = PTE_TO_PA(pte);
            pmem_free(pa, false); // 释放数据页（用户页）
        }
    }
}
}
}

```

设计要点：

- 使用递归处理三级页表结构
- **PTE_CHECK** 区分页表页和数据页
- 正确区分内核页和用户页

3.2.2 uvm_destroy_pgtbl()

销毁用户页表：

```

void uvm_destroy_pgtbl(pgtbl_t pgtbl)
{
    // 先解除特殊页的映射
    pte_t* pte_trampoline = vm_getpte(pgtbl, TRAMPOLINE, false);
    if (pte_trampoline)
        *pte_trampoline = 0;

    pte_t* pte_trapframe = vm_getpte(pgtbl, TRAPFRAME, false);
    if (pte_trapframe)
        *pte_trapframe = 0;

    // 递归销毁页表（从 level=2 开始）
    destroy_pgtbl(pgtbl, 2);

    // 释放顶级页表
    pmem_free((uint64)pgtbl, true);
}

```

设计要点:

- trapframe 和 trampoline 由内核管理, 不应被释放
- 先清除映射, 再递归销毁
- 最后释放顶级页表本身

3.2.3 uvm_copy_pgtbl()

拷贝页表:

```
void uvm_copy_pgtbl(pgtbl_t old, pgctl_t new, uint64 heap_top,
                    uint32 ustack_pages, mmap_region_t* mmap)
{
    // Step 1: 代码段和数据段
    if (heap_top > PGSIZE) {
        copy_range(old, new, PGSIZE, heap_top);
    }

    // Step 2: 用户栈
    if (ustack_pages > 0) {
        uint64 ustack_begin = heap_top;
        uint64 ustack_end = heap_top + ustack_pages * PGSIZE;
        copy_range(old, new, ustack_begin, ustack_end);
    }

    // Step 3: mmap 区域
    mmap_region_t* tmp = mmap;
    while (tmp != NULL) {
        uint64 begin = tmp->begin;
        uint64 end = begin + tmp->npages * PGSIZE;
        copy_range(old, new, begin, end);
        tmp = tmp->next;
    }
}
```

设计要点:

- 分三个部分分别拷贝
- 使用 copy_range 辅助函数处理连续区域
- 遍历 mmap 链表处理所有映射区域

3.2.4 uvm_mmap()

创建内存映射:

```

void uvm_mmap(uint64 begin, uint32 npages, int perm)
{
    if(npages == 0) return;
    assert(begin % PGSIZE == 0, "uvm_mmap: begin not aligned");

    proc_t* p = myproc();

    // 创建并插入 mmap_region
    mmap_region_t* new_region = mmap_region_alloc();
    new_region->begin = begin;
    new_region->npages = npages;
    new_region->next = p->mmap;
    p->mmap = new_region;

    // 分配物理页并建立映射
    for (uint32 i = 0; i < npages; i++) {
        uint64 va = begin + i * PGSIZE;
        uint64 pa = (uint64)pmem_alloc(false);
        assert(pa != 0, "uvm_mmap: out of memory");
        vm_mappages(p->pgtbl, va, pa, PGSIZE, perm | PTE_U);
    }
}

```

设计要点:

- 头插法插入 mmap 链表
- 逐页分配物理内存
- 自动添加 PTE_U 用户态访问权限

3.2.5 uvm_munmap()

取消内存映射:

```

void uvm_munmap(uint64 begin, uint32 npages)
{
    if(npages == 0) return;
    assert(begin % PGSIZE == 0, "uvm_munmap: begin not aligned");

    proc_t* p = myproc();
    uint64 end = begin + npages * PGSIZE;

    // 处理与现有区域的重叠
    mmap_region_t** prev_ptr = &(p->mmap);
    mmap_region_t* curr = p->mmap;

    while (curr != NULL) {
        uint64 curr_begin = curr->begin;
        uint64 curr_end = curr_begin + curr->npages * PGSIZE;

        if (curr_end <= begin || curr_begin >= end) {

```

```

        // 无重叠, 继续下一个
        prev_ptr = &(curr->next);
        curr = curr->next;
        continue;
    }

    if (curr_begin >= begin && curr_end <= end) {
        // 完全包含, 删除整个区域
        *prev_ptr = curr->next;
        mmap_region_free(curr);
        curr = *prev_ptr;
    } else if (curr_begin < begin && curr_end > end) {
        // 中间释放, 需要分裂成两个区域
        uint32 npages_before = (begin - curr_begin) / PGSIZE;
        uint32 npages_after = (curr_end - end) / PGSIZE;

        curr->npages = npages_before;

        mmap_region_t* new_region = mmap_region_alloc();
        new_region->begin = end;
        new_region->npages = npages_after;
        new_region->next = curr->next;
        curr->next = new_region;

        prev_ptr = &(new_region->next);
        curr = new_region->next;
    } else if (curr_begin < begin) {
        // 释放后半部分
        curr->npages = (begin - curr_begin) / PGSIZE;
        prev_ptr = &(curr->next);
        curr = curr->next;
    } else {
        // 释放前半部分
        uint32 npages_keep = (curr_end - end) / PGSIZE;
        curr->begin = end;
        curr->npages = npages_keep;
        prev_ptr = &(curr->next);
        curr = curr->next;
    }
}

// 尝试合并相邻区域
curr = p->mmap;
while (curr != NULL && curr->next != NULL) {
    uint64 curr_end = curr->begin + curr->npages * PGSIZE;
    if (curr_end == curr->next->begin) {
        mmap_merge(curr, curr->next, true);
    } else {
        curr = curr->next;
    }
}

// 解除页表映射
vm_unmappages(p->pgtbl, begin, npages * PGSIZE, true);
}

```

设计要点:

- 处理四种重叠情况：无重叠、完全包含、部分重叠、中间释放
- 区域分裂时需要分配新节点
- 合并相邻区域以减少碎片
- 最后统一释放物理页

3.2.6 堆管理函数

uvm_heap_grow:

```
uint64 uvm_heap_grow(pgtbl_t pgtbl, uint64 heap_top, uint32 len)
{
    uint64 new_heap_top = heap_top + len;
    assert(new_heap_top < TRAPFRAME, "heap grows too large");

    uint64 old_top_aligned = PG_ROUND_UP(heap_top);
    uint64 new_top_aligned = PG_ROUND_UP(new_heap_top);

    if (new_top_aligned > old_top_aligned) {
        uint32 npages = (new_top_aligned - old_top_aligned) / PGSIZE;
        for (uint32 i = 0; i < npages; i++) {
            uint64 va = old_top_aligned + i * PGSIZE;
            uint64 pa = (uint64)pmem_alloc(false);
            assert(pa != 0, "out of memory");
            memset((void*)pa, 0, PGSIZE);
            vm_mappages(pgtbl, va, pa, PGSIZE, PTE_R | PTE_W | PTE_U);
        }
    }

    return new_heap_top;
}
```

uvm_heap_ungrow:

```
uint64 uvm_heap_ungrow(pgtbl_t pgtbl, uint64 heap_top, uint32 len)
{
    uint64 new_heap_top = heap_top - len;
    assert(new_heap_top >= PGSIZE, "heap shrinks too much");

    uint64 new_top_aligned = PG_ROUND_UP(new_heap_top);
    uint64 old_top_aligned = PG_ROUND_UP(heap_top);

    if (new_top_aligned < old_top_aligned) {
        uint64 release_size = old_top_aligned - new_top_aligned;
        vm_unmappages(pgtbl, new_top_aligned, release_size, true);
    }
}
```

```
    return new_heap_top;
}
```

3.2.7 数据拷贝函数

uvm_copyin:

```
void uvm_copyin(pgtbl_t pgtbl, uint64 dst, uint64 src, uint32 len)
{
    while (len > 0) {
        uint64 va = PG_ROUND_DOWN(src);
        pte_t* pte = vm_getpte(pgtbl, va, false);
        assert(pte != NULL && (*pte & PTE_V), "invalid page");

        uint64 pa = PTE_TO_PA(*pte);
        uint64 n = PGSIZE - (src - va);
        if (n > len) n = len;

        memmove((void*)dst, (void*)(pa + (src - va)), n);

        len -= n;
        dst += n;
        src += n;
    }
}
```

uvm_copyout:

```
void uvm_copyout(pgtbl_t pgtbl, uint64 dst, uint64 src, uint32 len)
{
    while (len > 0) {
        uint64 va = PG_ROUND_DOWN(dst);
        pte_t* pte = vm_getpte(pgtbl, va, false);
        assert(pte != NULL && (*pte & PTE_V), "invalid page");

        uint64 pa = PTE_TO_PA(*pte);
        uint64 n = PGSIZE - (dst - va);
        if (n > len) n = len;

        memmove((void*)(pa + (dst - va)), (void*)src, n);

        len -= n;
        dst += n;
        src += n;
    }
}
```

uvm_copyin_str:

```

void uvm_copyin_str(pgtbl_t pgtbl, uint64 dst, uint64 src, uint32 maxlen)
{
    char* dst_ptr = (char*)dst;
    bool found_null = false;

    while (maxlen > 0 && !found_null) {
        uint64 va = PG_ROUND_DOWN(src);
        pte_t* pte = vm_getpte(pgtbl, va, false);
        assert(pte != NULL && (*pte & PTE_V), "invalid page");

        uint64 pa = PTE_TO_PA(*pte);
        uint64 n = PGSIZE - (src - va);
        if (n > maxlen) n = maxlen;

        char* src_ptr = (char*)(pa + (src - va));
        for (uint64 i = 0; i < n; i++) {
            *dst_ptr = *src_ptr;
            if (*src_ptr == '\0') {
                found_null = true;
                break;
            }
            dst_ptr++;
            src_ptr++;
            maxlen--;
            src++;
        }
    }

    // 确保字符串以 '\0' 结尾
    if (!found_null && maxlen == 0) {
        *(char*)(dst + maxlen - 1) = '\0';
    }
}

```

3.3 实现系统调用 (syscall.c 和 sysfunc.c)

3.3.1 syscall() - 系统调用分发器

```

void syscall()
{
    proc_t* p = myproc();
    uint64 num = p->tf->a7; // 系统调用号在 a7 寄存器

    if (num > 0 && num < sizeof(syscalls)/sizeof(syscalls[0]) &&
        syscalls[num]) {
        p->tf->a0 = syscalls[num](); // 返回值存入 a0
    } else {
        printf("syscall: unknown syscall number %d\n", num);
        p->tf->a0 = -1;
    }
}

```

```
}  
}
```

设计要点:

- 从 a7 寄存器读取系统调用号
- 通过函数指针数组分发调用
- 返回值自动存入 a0 寄存器

3.3.2 sys_brk() - 堆管理

```
uint64 sys_brk()  
{  
    proc_t* p = myproc();  
    uint64 new_heap_top;  
  
    arg_uint64(0, &new_heap_top);  
  
    if (new_heap_top == 0) {  
        return p->heap_top; // 查询当前堆顶  
    }  
  
    uint64 old_heap_top = p->heap_top;  
  
    if (new_heap_top < old_heap_top) {  
        // 收缩堆  
        uint64 diff = old_heap_top - new_heap_top;  
        p->heap_top = uvm_heap_ungrow(p->pgtbl, old_heap_top, diff);  
    } else if (new_heap_top > old_heap_top) {  
        // 扩展堆  
        uint64 diff = new_heap_top - old_heap_top;  
        p->heap_top = uvm_heap_grow(p->pgtbl, old_heap_top, diff);  
    }  
  
    return p->heap_top;  
}
```

功能:

- 参数为 0: 查询当前堆顶
- 参数 > heap_top: 扩展堆
- 参数 < heap_top: 收缩堆

3.3.3 sys_mmap() - 内存映射

```

uint64 sys_mmap()
{
    proc_t* p = myproc();
    uint64 start;
    uint32 len;

    arg_uint64(0, &start);
    arg_uint32(1, &len);

    // 参数检查
    if (len == 0 || len % PGSIZE != 0) {
        return -1;
    }

    uint32 npages = len / PGSIZE;

    if (start == 0) {
        // 自动分配地址
        start = p->heap_top + p->ustack_pages * PGSIZE;

        // 检查与现有 mmap 区域的冲突
        mmap_region_t* curr = p->mmap;
        while (curr != NULL) {
            uint64 curr_end = curr->begin + curr->npages * PGSIZE;
            if (start < curr_end && start + len > curr->begin) {
                start = curr_end;
            }
            curr = curr->next;
        }

        if (start + len >= TRAPFRAME) {
            return -1; // 地址空间不足
        }
    } else {
        // 检查地址对齐和冲突
        if (start % PGSIZE != 0) {
            return -1;
        }

        mmap_region_t* curr = p->mmap;
        while (curr != NULL) {
            uint64 curr_end = curr->begin + curr->npages * PGSIZE;
            if (start < curr_end && start + len > curr->begin) {
                return -1; // 冲突
            }
            curr = curr->next;
        }
    }

    // 执行映射
    uvm_mmap(start, npages, PTE_R | PTE_W);

    return start;
}

```

功能:

- start = 0: 自动分配地址
- start ≠ 0: 使用指定地址
- 返回映射区域的起始地址

3.3.4 sys_munmap() - 取消映射

```
uint64 sys_munmap()
{
    proc_t* p = myproc();
    uint64 start;
    uint32 len;

    arg_uint64(0, &start);
    arg_uint32(1, &len);

    // 参数检查
    if (len == 0 || len % PGSIZE != 0 || start % PGSIZE != 0) {
        return -1;
    }

    uint32 npages = len / PGSIZE;

    // 验证区域是否已映射
    mmap_region_t* curr = p->mmap;
    bool found = false;

    while (curr != NULL) {
        uint64 curr_end = curr->begin + curr->npages * PGSIZE;
        if (start < curr_end && start + len > curr->begin) {
            found = true;
            break;
        }
        curr = curr->next;
    }

    if (!found) {
        return -1;
    }

    // 执行取消映射
    uvm_munmap(start, npages);

    return 0;
}
```

3.4 修改进程结构

在 `proc.h` 中添加 `mmap` 字段:

```
typedef struct proc {
    int pid;
    pgtbl_t pgtbl;
    uint64 heap_top;
    uint64 ustack_pages;
    trapframe_t* tf;
    uint64 kstack;
    context_t ctx;

    struct mmap_region* mmap; // 新增: mmap 区域链表
} proc_t;
```

3.5 头文件修改

在 `vmem.h` 中添加前向声明:

```
// 前向声明, 避免循环依赖
typedef struct mmap_region mmap_region_t;
```

四、实验结果与测试

4.1 编译结果

```
$ cd whu-oslab-lab5/code
$ make clean && make
```

编译成功, 无错误和警告。

4.2 功能测试

4.2.1 mmap 仓库测试

使用 `mmap_show_mmaplist()` 可以查看仓库状态:

- 初始化后有 256 个节点可用
- 分配后节点数量减少
- 释放后节点数量恢复

4.2.2 堆管理测试

通过 `sys_brk` 系统调用：

- 扩展堆：分配新页面
- 收缩堆：释放多余页面
- 查询堆顶：返回当前值

4.2.3 mmap/munmap 测试

测试场景：

1. 自动分配地址映射
2. 指定地址映射
3. 部分取消映射
4. 区域分裂和合并

4.2.4 数据拷贝测试

使用 `sys_copyin`、`sys_copyout`、`sys_copyinstr` 测试：

- 跨页边界拷贝
- 字符串拷贝（遇 `\0` 终止）
- 用户态与内核态数据交互

五、实验心得

5.1 技术收获

1. 内存管理理解加深

- 理解了虚拟内存的分层管理
- 掌握了页表的递归操作
- 了解了内存映射的实现原理

2. 数据结构应用

- 链表管理 mmap 区域
- 对象池（仓库）模式的应用
- 指针操作和结构体嵌套

3. 系统调用机制

- 理解了系统调用的完整流程
- 掌握了参数传递和返回值处理
- 了解了用户态与内核态的数据交互

4. 并发安全

- 使用自旋锁保护共享数据
- 理解了临界区的保护机制

5.2 遇到的问题及解决

问题 1： 页表销毁时如何区分页表页和数据页？

解决： 使用 `PTE_CHECK(pte)` 宏判断，如果 `R|W|X` 位全为 0，则是页表页；否则是数据页。

问题 2： `munmap` 如何处理部分释放的情况？

解决： 实现了四种情况的处理逻辑：

- 无重叠：跳过
- 完全包含：删除
- 部分重叠：调整大小
- 中间释放：分裂成两个区域

问题 3： 如何避免 `trapframe` 和 `trampoline` 被错误释放？

解决： 在 `uvm_destroy_pgtbl` 中先清除这两个页的映射，再递归销毁其他页表。

问题 4： `copyin/copyout` 如何处理跨页数据？

解决： 使用循环，每次处理当前页内的数据，然后移动到下一页，直到完成全部拷贝。

问题 5： 类型依赖问题（`vmem.h` 需要 `mmap_region_t` 类型）

解决：使用前向声明（forward declaration）：

```
typedef struct mmap_region mmap_region_t;
```

5.3 改进方向

1. 更智能的地址分配

- 当前使用简单的顺序扫描
- 可以实现更复杂的首次适应、最佳适应算法

2. 内存碎片整理

- 当前只在 `munmap` 时合并相邻区域
- 可以实现周期性的碎片整理

3. 权限管理

- 当前 `mmap` 使用固定权限（`R|W`）
- 可以支持可执行权限、只读权限等

4. 错误处理

- 当前很多函数使用 `assert` 和 `panic`
- 可以实现更优雅的错误返回机制

5. 性能优化

- `mmap` 区域链表可以使用红黑树等高效数据结构
- 页表操作可以使用批量映射优化

六、实验总结

本次实验完整实现了用户虚拟内存管理系统，包括：

1. **mmap 区域管理**：实现了对象池模式的内存区域分配器
2. **页表操作**：实现了页表的创建、拷贝、销毁等完整生命周期管理
3. **堆管理**：实现了动态堆空间的扩展和收缩

4. **内存映射**：实现了完整的 mmap/munmap 系统调用

5. **数据拷贝**：实现了用户态与内核态之间的安全数据传输

通过本次实验，深入理解了操作系统内存管理的核心机制，掌握了虚拟内存、页表、系统调用等关键技术，为后续实现更复杂的操作系统功能打下了坚实基础。

实验过程中遇到了多个技术难点，通过仔细分析原理、参考文档、调试代码，最终都得到了解决。这不仅提升了编程能力，更培养了分析问题、解决问题的能力。

操作系统是一个复杂而精巧的系统，每一个细节都需要仔细考虑。本次实验让我对"细节决定成败"这句话有了更深的体会。